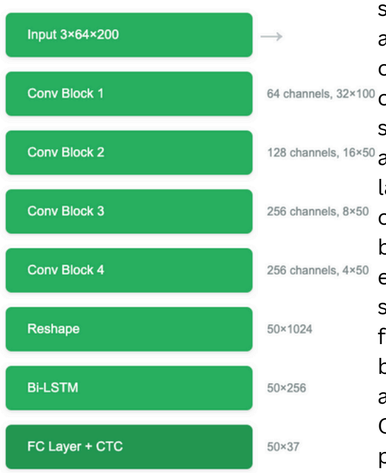
CS4243 MINI PROJECT ON CAPTCHA

By group 13: Jian Yi, Song Mengfei, Zhang Leran, Ma Yuehan

NUS National University of Singapore

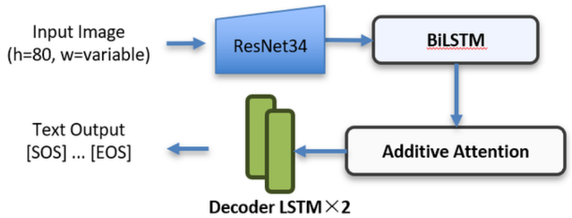
Baseline Model

Architecture:



The baseline model adopts a straightforward convolutional architecture. Its CNN backbone consists of four conventional convolutional blocks, each using standard 3*3 convolutions, ReLU activations, and max-pooling layers. This structure resembles classical **VGG-style** networks. The baseline system then feeds the extracted feature sequence to a single-layer bidirectional LSTM followed by a linear classifier. The baseline architecture represents a minimal and conventional CRNN pipeline suitable for performance benchmark

Whole Image Recognizer



1) Input & Preprocess:

- **Images**: filename → label; charset 0–9, a–z + <sos>/<eos>/<pad>.
- **Resize**: H=80, keep aspect, normalize

2) CNN → Sequence (Encoder)

- Truncate **ResNet-34** at layer3
- Apply **Project** + **LayerNorm** for the RNN
- **BiLSTM** encoder to fuse **left–right** context

2) Decoder

- **Attention**
  - Get a soft alignment over width
  - Produces a context vector
- **2-layer LSTM**
  - **Layer 1 (alignment-aware):** ingests visual context + previous token.
  - **Layer 2 (language-focused):** refines sequence regularities and disambiguates look-alike chars.

3) Training Loop

- **Teacher forcing (TF) annealing**
  - **Early epochs**: TF=1.0
  - Then linearly reduce to ~0.2
- **Optimization**
  - **Encoder:** smaller LR to stabilize fine-tuning
  - **Decoder:** higher LR to learn fast.

Model	Char_acc	Seq_acc
Baseline	72.27%	35.25%
Whole Image Recognizer	86.96%	56.30%
Char-Level recognizer	**87.09%**	**56.80%**

Generation

Fast, deterministic, interpretable; uses real character assets.

Learns data distribution; generates character variations; don't need exhaustive asset library

Generation Pipeline: Input String → Token Mapping → **Asset Sampling / cGAN Generation** → Augmentation → Composition → Occlusions → Output

Token Mapping: Maps each character to dataset tokens (e.g., 'A' → 'A_cap', '5' → '5')
Asset Sampling: Randomly selects real character images from folders of all characters pre-grouped into individual digits and individual capital and small letters

Character-Level Augmentation:

- Padding (25% horizontal, 5% vertical)
- Random rotation (±18°)
- Perspective warping (up to 12%)
- Gaussian blur (35% probability)

Line Composition:

- Random horizontal spacing (6-20px)
- Vertical jitter (-3 to +6px)

Canvas-Level Effects:

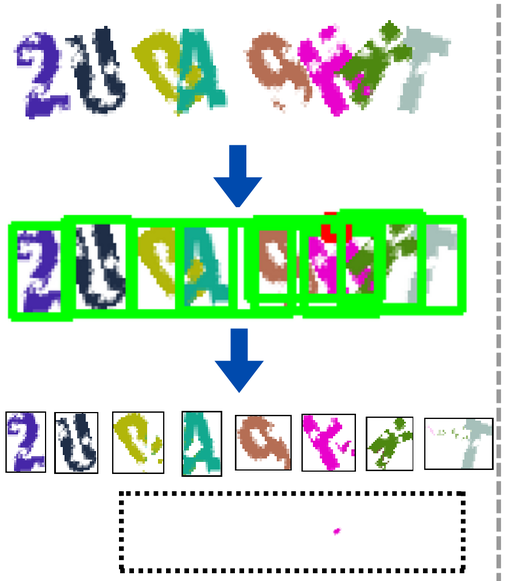
- Random line occlusions (60% probability)
- Random dot noise (50% probability)
- Final blur pass (25% probability)

Character-level Conditional GAN (cGAN) for CAPTCHA sprite generation
Generator: latent noise + class embedding → upsampling decoder
Discriminator: projection discriminator for class-aware realism scoring

Development Roadmap

Character-Level Recognizer

1. Segmentation Pipeline



- Background removal
- Assign contour ID
- Clustering with K-means, with the formula:

$$\arg\min_{S} \sum_{i=1}^{k} \sum_{x \in S_i} \sqrt{\begin{array}{c}(x_r - \mu_r)^2 + (x_g - \mu_g)^2 + (x_b - \mu_b)^2 + \\ \lambda(x_{dx} - \mu_{dx})^2 + \phi(x_{dy} - \mu_{dy})^2 + \omega \mathbf{1}(x_{dc}, \mu_{dc})^2\end{array}}$$

where k = len(Captcha) + 1; λ, ϕ and ω are penalties added to the difference in horizontal, verticle and contour ID, respectively.

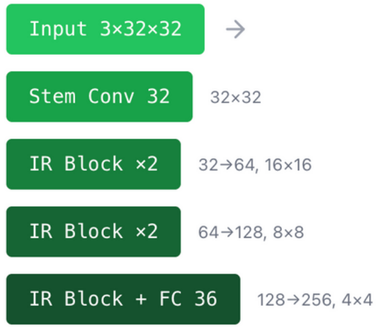
Optimal: λ=4.014, ϕ=0.100, ω=400.000

Adding spatial penalties to deal with the edge pixels that have close RGB. The left two are actually closer in RGB values!

- Noise removal at both character and CAPTCHA levels
- Compute bounding boxes
- Sort characters according to the leftmost pixel
- Crop tightly to remove extra pixels

2. Character Recognizer EfficientCaptchaCNN

Architecture:



Key Features:

- Depthwise separable convolutions
- Inverted residual blocks (MobileNetV2)
- Skip connections for gradient flow
- ReLU6 activation (mobile-optimized)